FEDEX DOMAIN EXPERT

Skills Suite

A reusable QA-pipeline skill suite for the PluginHive FedEx Shopify app.

Overview

This suite contains 14 skills that cover the full QA pipeline plus shared operators. The pipeline skills run in sequence (domain knowledge to AC to TC to AI QA to automation to handoff to sign-off); the operator skills are reusable access layers (Trello, Slack, Shopify, RAG, bug, knowledge).

Pipeline Skills

1. fedex-domain-core

When to use: Use when working inside the FedexDomainExpert project and the user asks anything about the PluginHive FedEx Shopify app, FedEx QA domain, app flows, FedEx carrier/API behavior, project architecture, or local RAG/code/wiki knowledge, or wants research-backed answers that may need browsing beyond the current knowledge base. This is the shared domain and research foundation that every other FedEx skill builds on.

The shared knowledge and research engine for the FedexDomainExpert project. Other skills

(AC writing, TC generation, AI QA, automation, handoff, sign-off) call into this skill

whenever they need authoritative facts about the app, the carrier, or the codebase.

When to use

- Any question about the PluginHive FedEx Shopify app, its UI flows, or settings
- Any question about FedEx carrier behavior, rate/label/return/manifest APIs
- Project architecture, file layout, or "where does X live" questions
- Requests that need current facts not already in the local knowledge base

Knowledge sources (search in this order)

1. **Local RAG —** `fedex_knowledge`: domain docs, wiki, app UI, FedEx API, approved cards
2. **Local RAG —** `fedex_code_knowledge`: automation POM, backend, frontend source
3. **Project files**: `CLAUDE.md`, `config.py`, `pipeline/`, `rag/` for ground truth
4. **Live web**: only when local knowledge is stale or missing, and say so explicitly

Rules

- Prefer local knowledge over memory. Quote file paths when you cite project facts.
- Never invent FedEx API field names, app routes, or button labels — look them up.
- When the knowledge base contradicts a memory or assumption, trust the knowledge base.
- If you browse the web, state which claims came from outside the local KB.
- Keep answers concise and decision-ready; link to the source rather than pasting it whole.

Output

A direct, sourced answer. When the answer feeds another skill (AC, TC, handoff), return the facts as clean structured notes the next skill can consume.

2. fedex-ac-writer-reviewer

When to use: Use when working inside the FedexDomainExpert project and the user gives a Trello card, feature request, bug or customer issue, PR note, or rough requirement and wants a dashboard-style User Story plus Acceptance Criteria generated, reviewed, rewritten, checked for toggle prerequisites, and prepared for Trello comment posting. Generation and review only — does not run AI QA or write test cases.

Turn a raw card or request into a clean, dashboard-style **User Story + Acceptance Criteria**, review it, and prepare it for posting back to Trello.

First reads (REQUIRED)

1. Use `fedex-domain-core` for any app/carrier facts the AC depends on.
2. Use `fedex-trello-operator` to fetch the card description, comments, and attachments when a card id/URL is given.

Inputs

- A Trello card (id/URL), feature request, bug report, PR note, or freeform requirement.
- Optional: related backlog cards, PR/code references, customer/Zendesk context.

Steps

1. **Research first** — pull card text, related backlog cards, PR/code refs, wiki, and customer issues via `fedex-domain-core`. Do not write AC from the title alone.
2. **Draft the User Story** — "As a <role>, I want <capability>, so that <value>."
3. **Draft Acceptance Criteria** — numbered, testable, covering happy path, edge cases, and negative cases. Each criterion must be independently verifiable.
4. **Toggle / prerequisite check** — detect any feature flag, setting, or store precondition the feature needs, and call it out explicitly.
5. **Review pass** — critique the draft for gaps, ambiguity, and untestable wording; auto-rewrite weak output and surface the review findings.
6. **Prepare for Trello** — format as a comment-ready markdown block. Post only with explicit user approval via `fedex-trello-operator`.

Rules

- Use only facts from research; never invent API fields, routes, or toggles.
- AC must be testable — no vague verbs like "should work properly".
- Keep US/AC in the project's dashboard format.

Output

US + AC markdown, the review findings, detected toggles, and a comment-ready block.

With explicit approval and Slack credentials, can send the standard toggle-enable DM to the designated owner via `fedex-slack-operator`.

3. fedex-dashboard-tc-publisher

When to use: Use when working inside the FedexDomainExpert project and the user already has a User Story plus Acceptance Criteria and now wants dashboard-style QA test cases generated from that US/AC, in the two dashboard publish formats — compact Trello QA comment and positive-case CSV rows for the Ai sheet tab. Generation-only; must not call Trello, Google Sheets, Slack, Shopify, or project LLM APIs directly. For detailed browser-executable test cases, use fedex-ai-qa-testcase-prep instead.

Generate QA test cases from approved US/AC in the two formats the dashboard publishes:

a **compact Trello QA comment** and **positive-case CSV rows** for the "Ai" sheet tab.

First reads

1. Use `fedex-domain-core` for app flows, expected behavior, and field-level detail.
2. Reuse known app navigation and evidence strategies when describing steps.

Inputs

- Approved User Story + Acceptance Criteria (required).
- Optional: card metadata, related flows.

Steps

1. Derive test cases from each acceptance criterion — cover positive paths primarily, plus key edge/negative cases where the AC calls for them.
2. **Trello comment format** — compact, scannable: TC id, title, type, priority, short steps.
3. **CSV format** — positive-case rows matching the "Ai" sheet columns exactly.
4. Keep any internal execution-flow metadata (manual/auto) out of both published formats.

Rules

- **Generation only.** Do NOT call Trello, Google Sheets, Slack, Shopify, or LLM APIs from here.
- Do not include internal-only fields in the published output.
- Match the dashboard's exact column order and comment layout.

Output

Two artifacts: the compact Trello QA comment block and the positive-case CSV rows.

Hand off to `fedex-trello-operator` / the sheet publisher only when the user explicitly asks to post.

4. fedex-ai-qa-testcase-prep

When to use: Use when working inside the FedexDomainExpert project and the user gives a Trello card link, card id, feature request, AC draft, or story and wants full, detailed, browser-testable test cases prepared specifically for the AI QA Agent / Chrome verification, using the same FedEx Domain Expert rules, evidence strategies, and automation-flow knowledge as the dashboard. Do not use for compact Trello comments, CSV rows, or Google Sheet formats — use fedex-dashboard-tc-publisher for those.

Produce **detailed, browser-executable** test cases designed for the AI QA Agent (Chrome / Computer Use) to run against the real Shopify + FedEx app.

First reads

1. Use `fedex-domain-core` for exact app flows, routes, and expected behavior.
2. Reuse the project's known navigation map and the five evidence strategies (label badge, download documents ZIP, request/response JSON, rate logs, printed-document codes).

Inputs

- Trello card (id/URL), feature request, AC draft, or story description.

Steps

1. Parse the feature into discrete, verifiable test cases.
2. For each TC, write explicit browser steps: navigate → set up order/settings → act → verify.
3. Specify the **order decision** (create_new / create_bulk / existing_fulfilled / existing_unfulfilled / none) and any preconditions/cleanup.
4. Specify the **evidence strategy** and exact pass/fail signal for each verification.
5. Mark each TC's execution flow as `manual` or `auto` (internal metadata).

Rules

- Steps must be concrete enough for a browser agent to execute without guessing.
- Never break or pollute existing store/app state — use safe test data.
- Reuse automation/codebase/domain knowledge before describing any navigation.

Output

A set of detailed, browser-testable TCs (with preconditions, steps, evidence strategy, and expected result) ready to feed `fedex-ai-qa-browser`.

5. fedex-ai-qa-browser

When to use: Use when working inside the FedexDomainExpert project to verify dashboard-generated FedEx Shopify app test cases in a real browser with Computer Use, following the same AI QA Verifier knowledge as the dashboard — parse TC metadata, reuse automation/codebase/domain knowledge before navigating, safely drive Shopify and FedEx app flows, ask QA only when truly blocked, and return pass/fail evidence without breaking the current store or app state.

Execute prepared test cases against the live Shopify admin + embedded FedEx app using a real browser (Computer Use), and return pass/fail with evidence.

First reads

1. Use `fedex-domain-core` for app structure and expected behavior.
2. Use `fedex-shopify-store-actions` to create/find the orders a TC needs.
3. Know the iframe model: app sidebar (Shipping, Settings, Products, PickUp, Rates Log) lives INSIDE `iframe[name="app-iframe"]`; Shopify Orders/Products live OUTSIDE it.

Inputs

- One or more prepared test cases (ideally from `fedex-ai-qa-testcase-prep`) with order decision, preconditions, steps, and evidence strategy.

Steps

1. Parse TC metadata (id, type, order decision, execution flow, evidence strategy).
2. Resolve preconditions (product flags, settings, order setup) deterministically first.
3. Drive the flow: manual vs auto label launch, sidedock config, settings, manifest, etc.

4. Capture evidence using the right strategy (label badge, Download Documents ZIP, How To → request/response JSON, Rates Log, Print Documents codes).
5. Summarize raw payloads into compact business facts before judging.
6. Return a verdict: **pass / fail / partial** with the supporting evidence.

Rules

- Never corrupt store/app state; reset any toggles you changed.
- Use deterministic orchestration where flows are known; fall back to the agentic loop only when needed.
- Ask QA a specific question only when genuinely blocked — never guess a destructive action.
- Honor stop requests at the next safe checkpoint.

Output

Per-TC verdict (pass/fail/partial) + evidence (JSON facts, screenshots, document codes), ready for bug review or sign-off.

6. fedex-automation-writer

When to use: Use when working inside the FedexDomainExpert project after US/AC generation, dashboard TC generation, and AI QA browser verification are complete, and the user wants Playwright TypeScript automation written for a FedEx Shopify card. Reuse the dashboard automation conventions, inspect Chrome manually for DOM/locators, reuse existing automation locators and POM methods first, create new locators only when missing, and use saved AI QA locator traces by card id/name when available.

Write **Playwright TypeScript** automation for an approved, AI-QA-verified FedEx Shopify card, matching the existing automation codebase conventions.

First reads

1. Use `fedex-domain-core` and the code RAG for POM structure, fixtures, and helpers.
2. Look up saved AI QA locator traces by card id/name when they exist.
3. Read the existing automation codebase (`AUTOMATION_CODEBASE_PATH`) before writing anything.

Inputs

- Approved TCs + AI QA evidence/locator traces for the card.

Steps

1. Map each TC to a spec. Reuse existing POM methods and locators **first**.
2. Inspect the live DOM in Chrome to confirm selectors before coding new ones.
3. Create new locators/POM methods only where none exist; follow existing naming and file layout.
4. Use project fixtures for store/order setup (e.g. the Shopify order uploader) rather than ad-hoc setup.
5. Keep specs deterministic and independent; avoid `networkidle` on Shopify pages.

Rules

- Reuse before you create — do not duplicate existing locators or helpers.
- Match the codebase's TypeScript style, folder structure, and POM patterns exactly.
- All paths come from env (`AUTOMATION_CODEBASE_PATH`, etc.) — never hardcode machine paths.

Output

Playwright TS spec(s) + any new POM methods/locators, placed to match the existing automation project structure, ready to run.

7. fedex-handoff-docs

When to use: Use when working inside the FedexDomainExpert project after cards are approved and the user wants professional release handoff documents like the dashboard Handoff Docs tab — Support Guide, Business Brief, or both — generated from approved US/AC, TCs, AI QA evidence, release and card metadata, toggles, and member ownership. If the user requests only one document, generate only that document and its PDF.

Generate professional release handoff documents for approved FedEx Shopify cards: a **Support Guide** (support/demo team) and/or a **Business Brief** (non-technical stakeholders), each as markdown + PDF.

First reads (REQUIRED)

1. Use `fedex-trello-operator` to fetch card details, members, and approval metadata.
2. Use `fedex-domain-core` for accurate app navigation, button names, and expected behavior.
3. For "Where to Find" and "Walkthrough" sections, use the known app navigation map — do NOT infer UI steps from AC text.

Inputs

- Approved card(s): US/AC, reviewed TCs, AI QA summary/evidence, release name, toggles, members.

Steps

1. Build a context object per card (description, AC, TCs, QA evidence, toggles, ownership).
2. **Support Guide** — title + Trello link, feature summary, toggles/prerequisites, where to find, step-by-step walkthrough, expected behavior. (Per current project rules, omit Developed/Tested By, Business-Safe Explanation, Common Questions/Troubleshooting, and Known Limitations.)
3. **Business Brief** — plain-English value: problem, what's new, who benefits, why it matters, availability. No QA/dev attribution, no jargon.
4. Render markdown → PDF. For multi-card releases, combine into one document with a cover/TOC.
5. On request, attach the PDF to Trello or upload to Slack (channel or DM) after explicit approval.

Rules

- Generate only the document(s) the user asked for — do not default to both.
- Use only facts from context; mark unknowns rather than inventing release numbers/toggles.
- Do not emit "QA NOTE / Navigation Confirmation Needed" blocks; write the best supportable steps.

Output

The requested handoff document(s) as markdown + branded PDF, ready to share.

8. fedex-signoff-message

When to use: Use inside the FedexDomainExpert project when QA asks to prepare or send the final QA sign-off message for a Trello release line or list. Fetch all cards from the line, prepare the dashboard-style Slack sign-off message, ask QA for any Backlog bug links if bugs were created, review the message with QA, and send to the Slack channel only after QA provides the channel and explicitly confirms.

Prepare and (after explicit confirmation) send the final QA **sign-off message** for a release line to Slack, in the dashboard format.

First reads

1. Use `fedex-trello-operator` to fetch every card in the release line/list.
2. Use `fedex-slack-operator` to send — only after explicit channel + confirmation.

Inputs

- The Trello release line/list (name or id).
- Optional: Backlog bug links for issues found during the cycle.

Steps

1. Fetch all cards in the line; collect titles, ids, and status.
2. Build the dashboard-style sign-off summary (release name, cards covered, QA result).
3. If bugs were created, ask QA for the Backlog bug links and include them.
4. **Review with QA** — show the drafted message and wait for edits/approval.
5. Send to Slack **only** after QA names the channel and explicitly confirms.

Rules

- Never send without an explicit channel + explicit "send it" confirmation.
- Don't fabricate results — reflect the actual QA verdicts and any open bugs.

Output

The reviewed sign-off message, and (on confirmation) confirmation that it was posted to the named Slack channel.

Operator Skills

9. fedex-bug

When to use: Use when working inside the FedexDomainExpert project and QA reports a bug during AI QA, browser, or manual testing and wants it formatted, checked against existing Trello Backlog cards, and created in the Trello Backlog list. Mirrors the dashboard Bug Reporter flow — plain-English QA issue to Jira-style bug draft to duplicate check to Backlog card creation after approval.

Turn a plain-English QA issue into a structured bug, check for duplicates, and create it in the Trello Backlog list — mirroring the dashboard Bug Reporter.

First reads

1. Use `fedex-domain-core` to confirm expected vs actual behavior when unclear.
2. Use `fedex-trello-operator` to search the Backlog and to create the card.

Inputs

- A plain-English description of the bug (ideally with steps, evidence, environment).

Steps

1. Draft a **Jira-style bug**: title, environment, steps to reproduce, expected, actual, severity, evidence/attachments.

2. Duplicate check — search existing Backlog cards; if a likely duplicate exists, surface it and ask before creating a new one.

3. On approval, create the card in the **Backlog** list via `fedex-trello-operator`, with appropriate labels/severity.

Rules

- Always run the duplicate check before creating.
- Never create the card without explicit approval.
- Keep severity honest and evidence concrete.

Output

The bug draft, duplicate-check result, and (after approval) the created Backlog card link.

10. fedex-trello-operator

When to use: Use inside the FedexDomainExpert project when the user asks to work with Trello using project .env credentials — read boards, lists, cards, descriptions, comments, checklists, attachments, fetch all cards from a list, identify the developer assigned to a card, add comments or QA replies, move cards, search cards, or create generic Trello cards. For QA bug Backlog creation use fedex-bug. Requires explicit user intent before any Trello write.

Read and write Trello using the project's `.env` credentials (`TRELLO_API_KEY`, `TRELLO_TOKEN`, and an optional default board). The shared Trello access layer for the other FedEx skills.

Capabilities

- **Read**: boards, lists, cards (description, comments, checklists, attachments, members), fetch all cards in a list, identify the assigned developer, search cards on a board.
- **Write** (explicit intent required): add comments / QA replies, move cards between lists, create generic cards.

Rules

- Reads are free; **any write requires explicit user intent** ("post this comment", "move card X").
- Board access is workspace-aware: resolve boards, then lists for the selected board.
- Never invent card content; quote what's actually on the card.
- For creating QA bug cards in Backlog, defer to `fedex-bug` (it adds the duplicate check).

Output

The requested Trello data, or confirmation of the write performed (with the card/comment link).

11. fedex-slack-operator

When to use: Use inside the FedexDomainExpert project when the user asks to work with Slack using project .env credentials — search Slack users, list channels, fetch messages from a visible channel, read a thread, send a channel message, reply in a thread, send a DM by name or ID, or coordinate with Trello developer assignment. Requires explicit user intent before any Slack send.

Read and write Slack using the project's `.env` credentials. The shared Slack access layer for the other FedEx skills (sign-off, handoff uploads, toggle-enable DMs, developer notifications).

Capabilities

- **Read**: search users, list channels, fetch channel messages, read a thread.
- **Write** (explicit intent required): post a channel message, reply in a thread, send a DM by user name or ID, upload a file (e.g. a handoff PDF).

Rules

- Reads are free; **any send requires explicit user intent** naming the target (channel/user).
- Resolve user names to IDs before DMing; confirm the resolved person when ambiguous.
- Never send sign-off or developer messages without explicit confirmation.

Output

The requested Slack data, or confirmation of the message/upload sent (with destination).

12. fedex-shopify-store-actions

When to use: Use when the user wants to perform any Shopify Admin API action (REST or GraphQL) on a store — create/update/archive/delete products (simple, variable, up to 2048 variants via GraphQL), create/cancel/delete/update orders (preset, custom, draft, with stock bypass so test orders never deduct inventory), create order plus fulfillment plus tracking in one call, bulk inventory update, bulk cleanup by tag, update shipping address, manage customers, list fulfillments/carrier services/webhooks/metafields/collections/locations, or create refunds — all via natural language. The token comes from the automation .env automatically; if the store isn't found there, ask for a token.

Perform any Shopify Admin API action (REST or GraphQL) on a store from natural language.

Used by AI QA to set up test orders and by general store maintenance tasks.

Auth

- Read `STORE`, `SHOPIFY_ACCESS_TOKEN`, `SHOPIFY_API_VERSION` from the automation `.env`.
- If the requested store isn't configured there, ask the user for a token before proceeding.

Capabilities

- **Products**: create/update/archive/delete; simple, variable (up to 2048 variants via GraphQL), digital, dangerous.
- **Orders**: create (preset/custom/draft), cancel, delete, update; bypass stock so test orders never deduct real inventory; create order + fulfillment + tracking in one call.
- **Inventory**: bulk update across variants; bulk cleanup by tag.
- **Other**: update shipping address, manage customers, create refunds; list fulfillments, carrier services, webhooks, metafields, collections, locations.

Rules

- Default test orders to stock-bypass so inventory is never wrongly deducted.
- Confirm before destructive actions (delete/cancel/refund) unless clearly authorized.
- Never run a write against a store the user didn't name.

Output

Confirmation of the action with the resulting ids (product/order/fulfillment) and any tracking info.

13. fedex-rag-sync

When to use: Use inside the FedexDomainExpert project when the user asks to pull latest and sync or reindex RAG knowledge for codebase, automation, backend, frontend, wiki, Shopify Actions, or full knowledge. Backend syncs master, frontend syncs main, wiki uses source-only pull and reindex, and automation is branch-aware and must ask QA for the branch unless provided. Never run a full reindex unless explicitly requested.

Pull the latest source and reindex the project's RAG knowledge collections

(`fedex_knowledge`, `fedex_code_knowledge`).

Sources & branch rules

- **backend** → sync `master`
- **frontend** → sync `main`
- **automation** → branch-aware; **ask QA for the branch** unless it's provided
- **wiki** → source-only pull + reindex
- **shopify_actions** → preserve the exact configured path (may end in a trailing space)
- **codebase / full** → only on explicit request

Steps

1. Confirm which source(s) to sync; for automation, confirm the branch first.
2. Pull latest for each selected source on its correct branch.
3. Run the partial reindex for just those sources, e.g.:

```
PYTHONPATH=. .venv/bin/python ingest/run_ingest.py --sources wiki shopify_actions
```

4. Report what was pulled and reindexed.

Rules

- **Never run a full reindex unless explicitly requested** — prefer partial, scoped reindexes.
- Use env-driven paths only; never hardcode machine-specific folders.
- Always confirm the automation branch before syncing it.

Output

A summary of sources pulled, branches used, and collections reindexed.

14. fedex-knowledge-maintainer

When to use: Use inside the FedexDomainExpert project after a FedEx release card cycle is complete, or when QA asks to update old, wrong, missing, or outdated project knowledge. Updates approved-card RAG, QA retrospective feedback, durable AGENTS/skill rules, and replaces obsolete knowledge without duplicating stale instructions.

Keep the project's durable knowledge current after a release cycle: embed approved cards,

record QA retrospective feedback, and update long-lived rules — without leaving stale duplicates.

When to use

- A release card cycle finished and its learnings should be captured.
- Project knowledge is wrong, outdated, missing, or contradictory.

Steps

1. **Approved-card RAG** — embed approved cards' description/AC/TCs into `fedex_knowledge` so the system learns from each sprint.
2. **Retrospective feedback** — record QA notes on what the pipeline got right/wrong this cycle.
3. **Durable rules** — update `CLAUDE.md` / `AGENTS` / skill instructions when a rule genuinely changed.
4. **Replace, don't duplicate** — when fixing knowledge, edit or remove the obsolete entry rather than adding a competing one.

Rules

- Don't duplicate existing knowledge or rules — update in place.
- Only record durable, reusable facts; skip one-off conversation details.
- Verify a referenced file/flag still exists before writing a rule about it.

Output

A summary of what was embedded, what feedback was recorded, and which rules were updated or retired.